Basam

Your Phase 1 work and preparation guide

1. Your responsibility

Your area: driver management, immutable date/time, CSV integration and the command-line shell. You implement substantial registry algorithms and driver deletion as well as connecting input data to the system.

You primarily implement	What you deliver
Driver.java	Person subclass with vehicle plate/type, validation and ID-based comparison.
DriverList.java	Every driver-list operation: sorted insertion, unique ID/plate, search and removal of all matches.
DateTime.java	Immutable date/time with chronological comparison and formatting.
RideSharingSystem.java	Your eight methods listed on page 4, including driver deletion and both assigned loaders.
Main.java	Shared menu shell, automatic startup, options 5-7 and 0, and invalid-input handling. Other members implement their handlers.
VehicleType.java	Reuse the supplied enum unchanged; do not create another version.

Your reviewer: Waleed. **Integration tester for your area:** Nawaf. You review Nawaf's list/rider code and test Waleed's scheduling and ride behavior.

First actions:

1. Read IDriver, IDriverList, IDateTime and your IRideSharingSystem methods. Review inherited IPerson methods too.
2. Agree with Nawaf on Person constructors, the exact classroom list API and history behavior.
3. Agree with Waleed on DateTime construction, scheduling calls and the shared whole-ride cleanup helper.
4. Implement DateTime and driver models against the agreed skeletons; write parsing independently while scheduling is developed.

Important: owning the Main shell does not make you the sole integration owner. Everyone writes their system/menu methods, reviews code and helps fix the complete workflow.

Ordering question: DriverList must stay sorted by driver ID. The provisional alphabetical display solution is a second custom name-ordered index; confirm display expectations with the instructor. Keep both indexes consistent and retain the same Driver objects.

This guide follows the existing team split. It is a work checklist, not a claim that your code or tests are already finished. No schedule or effort estimates are included.

2. What you should study

Must know	Where you use it
Classes, constructors, inheritance and interfaces	Driver extends Person implements IDriver; DriverList implements IDriverList; DateTime implements IDateTime.
Getters, setters, private and final	Validate fields, inherit immutable person ID, keep all DateTime fields immutable.
Comparable and method overriding	compareTo(IDriver) compares IDs; compareTo(IDateTime) compares five fields chronologically.
Generics, references and null	LinkedList<IDriver>, LinkedList<IRide>; registry searches and driver histories.
Exact string matching and enums	Full names and plates; VehicleType.valueOf; SEDAN, LUXURY_SEDAN, SUV, VAN.
Exceptions	Invalid phone/plate setters throw IllegalArgumentException; Main handles input errors; loaders handle I/O/parse failures.
File reading and resource closing	BufferedReader/FileReader or equivalent, ideally try-with-resources. File input is allowed in loaders.
split, parseInt, arrays and int[]	CSV columns, date/time fields and semicolon-separated shared rider IDs.
Loops and conditionals	Registry traversal, all-match removal, driver cascades, menu loop and dispatch.

Your minimal linked-list checklist:

☐ Traverse safely and find a matching driver by ID or plate.

☐ Insert in ID order at beginning, middle or end.

☐ Collect every matching name/type into another custom list.

☐ Delete first, middle, last or only nodes; remove all repeated name/type matches.

☐ Maintain consistent counts and name/ID indexes.

☐ Copy a history before deleting rides from the original structure.

☐ Build result lists using O(1) append so copying stays O(n).

DateTime comparison order: year -> month -> day -> hour -> minute. Stop at the first differing field. Equal values return 0. Do not compare MM/DD/YYYY display strings to decide chronological order.

Useful while building: small regular expressions, StringBuilder/formatting, debugger/assertions and Git. Plate validation can use [A-Z]{3}[0-9]{4}; phone validation is inherited from Person.

Not required: recursion, linked-list reversal, circular/doubly linked lists, iterator implementation, streams, GUI or database work. Java collection structures are prohibited; ordinary arrays and a console Scanner are not collection structures.

Ready-to-start exercise: store drivers with IDs 20, 10, 30 in sorted order, reject a repeated plate with a different ID, and compare the final minute of one year with the first minute of the next.

3. Your class implementation checklist

Driver extends Person implements IDriver:

- Add vehiclePlate and VehicleType. Inherit ID, name, phone and history from Nawaf's Person class.
- Implement getVehiclePlate, setVehiclePlate, getVehicleType, setVehicleType and compareTo(IDriver).
- Validate exactly three uppercase letters followed by four digits; invalid values throw IllegalArgumentException before changing the old value.
- Enforce ID/plate uniqueness in DriverList/system, not in a standalone Driver setter that cannot inspect all drivers.
- Compare driver IDs with Integer.compare. Override toString to return useful fields without printing or expanding the entire ride history.

DriverList method	What your implementation must do
add	Reject duplicate ID or plate; insert by driver ID; update the agreed name index.
findById / findByVehiclePlate	Return one match or null under the agreed miss policy.
findByName / findByVehicleType	Collect every match in a new custom list. Names use exact full-name equality.
getAll	Return an ID-ordered shallow snapshot under the agreed contract.
removeById / removeByVehiclePlate	Remove at most one; return true/false; maintain both indexes.
removeByName / removeByVehicleType	Remove ALL matches and return their count. Do not skip adjacent matches.
size	Return the correctly maintained count.

DateTime implements IDateTime:

- Use final year, month, day, hour and minute fields, initialized only by the constructor.
- Implement getYear, getMonth, getDay, getHour, getMinute, format and compareTo(IDateTime).
- Validate the stated ranges; month-length/leap-year checking is useful extra robustness. Agree on invalid-input handling.
- Format values as MM/DD/YYYY HH:MM with zero padding. Constructor proposal: DateTime(year, month, day, hour, minute).
- Example: 10/28/2026 08:45 becomes DateTime(2026, 10, 28, 8, 45).

Do not modify VehicleType.java. LUXURY_SEDAN contains an underscore. Convert the trimmed CSV value with VehicleType.valueOf; reject unrecognized values.

Index maintenance: a driver's name change requires name-index repositioning; a stored plate change must preserve uniqueness. No edit menu is required, but setters alone do not coordinate registry rules. Use the team's controlled-update contract for any such edits.

4. Your system methods, CSV and Main

Implement these **eight existing IRideSharingSystem methods** using their exact supplied signatures.

Method	Your responsibility
loadDriversFromCSV	Parse each five-field row; construct Driver and call addDriver.
loadRidesFromCSV	Parse each seven-field row and call Waleed's appropriate scheduler. Enforce normal conflicts and history updates through it.
addDriver	Enforce unique driver ID and unique plate; return false for invalid/duplicate requests.
searchDriverById	Delegate to the unique-ID search.
searchDriverByVehiclePlate	Delegate to the unique-plate search.
searchDriversByVehicleType	Return every matching driver.
getAllDrivers	Return all drivers using the confirmed display order; use maintained name index if required.
removeDriver	Delete every assigned ride and all related history entries, then remove the driver.

Driver CSV columns: driverId, name, phoneNumber, vehiclePlate, vehicleType. Parse ID as int; preserve phone/plate as strings; convert vehicle type to the supplied enum.

Ride CSV columns: rideType, pickupLocation, pickupDateTime, dropoffDateTime, dropoffLocation, driverId, riderIds. Split riderIds by semicolon into int[]. PRIVATE has one rider; new SHARED bookings need multiple distinct riders. Parse dates into immutable DateTime objects.

All files have no header. Do not skip row 1. Validate exact field counts using split(",", -1). The supplied simple CSVs have no quoted embedded commas. Use existing registered people; do not create replacement Rider/Driver objects while loading rides.

Main startup: create one system, call Nawaf's loadRidersFromCSV, then loadDriversFromCSV, then loadRidesFromCSV before showing the menu. Check all return values. The provisional loader policy returns false at the first error while retaining prior accepted rows; Main must not announce complete success after a partial load.

Your menu work: shell and input recovery; 5 -> getAllDrivers; 6 -> searchDriversByVehicleType; 7 -> construct Driver then addDriver; 0 -> exit. Integrate Nawaf's 1-4 and Waleed's 8-12 in exactly that numbered order.

removeDriver flow: find driver or return false; snapshot their history; call the shared removeRideEverywhere helper on every assigned private/shared ride; then remove the driver from all driver indexes. Other drivers' rides stay. Coordinate the helper with Nawaf rather than writing incompatible cleanup logic.

5. Tests and Big-O you must deliver

Test	Expected result
Driver ordering	IDs 20, 10, 30 become 10, 20, 30.
Duplicate ID or plate	Reject without changing either index or count.
Repeated driver name/type	Allowed; search and bulk removal handle ALL matches.
Plate validation	ABC1234 accepted; abc1234, AB1234, ABC123X and null rejected; invalid setter preserves previous value.
DateTime comparisons	Correct result when only year/month/day/hour/minute differs; equal fields compare as 0.
Year boundary	12/31/2026 23:59 is earlier than 01/01/2027 00:00.
Empty/singleton and deletions	No null errors; correct links/counts; absent removal returns false/0.
Driver removal	All assigned private/shared rides disappear globally and from every participant history.
Unrelated records	Other drivers' rides remain; removing absent driver changes nothing.
Clean startup	100 riders, 30 drivers, 40 rides; 76 rider-history entries in total and 40 driver-history entries.
Bad CSV fixtures	Missing file, bad field count/enum/number/date, unknown person or conflict returns false under agreed policy.
Main input and menu	Exact menu order; 0 exits; invalid choices/numbers do not crash; no business-class console printing.

Supplied-data checks: 11 SEDAN, 7 SUV, 6 VAN and 6 LUXURY_SEDAN drivers. Searching full name Khalid Al-Harbi returns IDs 3 and 22. Phone strings retain leading zeros.

Your analysis	Expected bound under the agreed design
Driver registry add/search/remove/copy	O(d), with a constant number of scans and O(1) result append.
size; driver compareTo; DateTime compareTo	O(1); DateTime compares at most five fields.
Loading d drivers into an empty sorted list	O(d^2): sum of existing-list scan lengths.
Loading rides	Parsing plus the SUM of scheduling costs for every row; repeated sorted insertion can contribute a quadratic term.
Driver cascade	O(d + (a+P_a)*m + P_a) is a safe bound for repeated linear cleanup.

Here d is drivers, m is stored rides, a is rides assigned to the removed driver, and P_a their total rider memberships. Cleanup may scan global rides and each participant's history. Derive the exact bound from your final helper calls.

Analyze your constructors, getters/setters, formatting, loader helpers and Main too. Per menu choice, include the called method plus parsing/output. Do not call the entire interactive loop O(n) without defining how many choices occur. Your final report must show the loop counts/summations, not only Big-O labels.

6. Team handoffs and your completion checklist

Teammate	What you exchange
Nawaf	Receive Person, the custom list contract, rider searches/loading and shared whole-ride cleanup. Supply driver/time behavior and the Main shell. You review his lists/rider code; he tests your driver/date/loading integration.
Waleed	Supply immutable DateTime and driver existence/history behavior. Receive both schedulers, ride queries and ride subtype behavior. He reviews your code; you test his bookings, overlap rules and ride searches.

What you must test in Waleed's code: same driver with overlapping times must fail; exact adjacency must pass; one unavailable rider must reject an entire shared booking; rejected requests must not change any history. Run these through both CSV loading and the relevant scheduling method.

Group decisions to confirm: constructor parameter order; classroom list API; live history versus snapshot getters; filename/path handling; loader failure semantics; ID versus alphabetical display; and repeated shared pickup lookup. Main must be the only production console I/O class. File readers in loaders are still correct.

Demo practice:

- Explain why a valid plate format is different from a unique plate.
- Trace chronological comparison and explain why formatted date strings are unsuitable for it.
- Parse one shared CSV record and show how existing people are resolved.
- Trace driver removal into every affected rider history.
- Explain Nawaf's all-match deletion and Waleed's overlap test.

You are finished when:

☐ Driver, DriverList and DateTime compile against unchanged interfaces and the common Person/list code.

☐ All eight assigned system methods and your Main responsibilities work.

☐ Startup loads people before rides and does not skip a first record.

☐ Driver removal cleans global rides and every affected history.

☐ Every own-area test passes, and you have tested Waleed's scheduling.

☐ Waleed has reviewed your work; Nawaf has tested it; you have reviewed Nawaf's code.

☐ Your final Big-O section includes CSV and cascade helper costs, with derivations.

☐ Your actual coding/review/testing/integration work is recorded.

☐ You can explain the whole system and help verify the final source ZIP, report PDF and demo.

Submission reminder: the report needs all three names and actual student IDs, actual division of work, and worst-case method analysis. The full menu is a shared deliverable. All members must participate across the project and attend the required demonstration.

Sources: current CSC212_Phase1_Team_Roadmap.pdf, especially Sections 6-8, 10 and 14; supplied RideSharing_Assignment.pdf pp. 1-14; IDriver, IDriverList, IDateTime, IRideSharingSystem, VehicleType; drivers_30.csv and rides_40.csv. This guide preserves the agreed allocation.